Architecture

1. **Repository Architecture**

An example is the back-end workflow of an e-commerce site. It uses elements of OOP and database management, but it breaks down the exact files and directories, so it's flexible and can be modularized so that the architecture doesn't have to change when an element changes. This is perfect for handling inventory, including adding new classes and managing stock without more complex handling.

2. **Client-server Architecture**

Browsers like Chrome and Safari act as clients, sending requests to web servers to load pages. The servers then process the inputs and relay information back as needed. It would be most suitable when there is an input query (such as a search term) and when there is a need for scalability without compromising the server. This therefore makes it possible for as many people as want to use a particular web browser to be able to use it without the servers on the back end crashing.

3. **Pipe and Filter Architecture**

An example of this in a real-world scenario is the graphics rendering pipeline. Raw vertices and primitives are first transformed, then rasterized into fragments and then processed as fragments before being merged into an output. It would be most useful for sequentially processed data and the data source and sink do not need to know where the data is coming from or going to. This is useful for graphics rendering, because there is a clear and sequential pipeline from vertices and primitives to outputs.

- Give two examples of systems combining two or more patterns, and state the strengths and limitations of these combined systems.

Gmail is an example of layered architecture. There is the layer that processes the messages (like a pipe and filter), and then the layer that communicates with users (like a repository). There's also another layer that works with a database to pull specific users' messages and to manage users' interfaces so, for example, spam is diverted to a junk folder. The layers are autonomous, so you can add more and more languages and features but you don't interfere with the messaging feature, but upper you have to connect each layer in the upper network to each layer in lower networks – for example, first pull my inbox, then pull a particular message and then make sure it's in English.

Customer relationship management sites that work with multiple variables to sell or market items (like offering different hotels to different users depending on what their occasion is) are examples of data-centric architecture. There is an input for a search (like a client-server architecture), but then you have different repositories that are accessed and this requires back-end sorting. This is useful because it means the code

doesn’t need to change, but it might be prone to issues such as data quality in one area impacting the table of options or damage in one cell impacting the entire population.